Assignment 3: Due May 12th, 2026

Turn-in

This is an individual assignment. You are required to submit both the answers and code you used for this assignment on Gradescope, instructions on how to join the course on Gradescope and submit the assignment are posted at the end of this writeup. You may submit a PDF writeup along with code for your submission, or submit a jupyter notebook with the code + answers. Remember to fully evaluate the cells in your jupyter notebook if you choose that option for submission. Late submissions will be accepted and deducted in accordance to the course late-day policy.

Overview

In this assignment, you will infer Quality-of-Experience (QoE) metrics for a Google Meet video conferencing application from a given packet capture. The pcap file is available for download here.

The goal of this assignment is to give you practice programatically analyzing packet captures for real-time multimedia applications. Unlike video streaming applications such as YouTube —which use adaptive bitrate (ABR) streaming over HTTP—video conferencing applications like Google Meet use the **Real-time Transport Protocol (RTP)** to transmit audio and video data over UDP. You will learn how to identify individual RTP streams, reconstruct video frames from RTP packets, and compute QoE metrics such as frames per second and jitter.

Getting Started

Before you begin analysis, you will need to convert the pcap file into a CSV format. Working directly with the pcap binary format requires specialized libraries and more complex code, whereas a CSV gives us a simple, tabular representation of each packet's fields that we can easily load and manipulate with standard tools like Python's `csv` module or `pandas`. Note that CSV is not even the most optimal format for processing large amounts of network data; formats like Parquet offer better performance and compression, but CSV provides ease-of-use since the data is stored as human-readable text that you can inspect and debug at a glance.

To extract RTP fields from a pcap into a CSV, you can use `tshark`, the command-line version of Wireshark. The `-T fields` option tells tshark to output specific fields, and you use `-e` to specify each field you want to extract. For RTP analysis, you will need the following fields:

```
tshark -r google_meet.pcap -T fields -E separator=, -E header=y \
  -e frame.number -e frame.time_epoch -e ip.src -e ip.dst \
  -e udp.srcport -e udp.dstport -e udp.len \
  -e rtp.ssrc -e rtp.timestamp -e rtp.seq \
  -e rtp.cc -e rtp.ext -e rtp.ext.len \
  > google_meet.csv
```

Some useful tshark field names for RTP analysis include:

- `frame.number` — the packet number in the capture

- `frame.time_epoch` — the timestamp of the packet (in seconds since epoch)
- `ip.src` / `ip.dst` — source and destination IP addresses
- `udp.len` — the total UDP datagram length in bytes, including the 8-byte UDP header
- `rtp.ssrc` — the Synchronization Source identifier, which uniquely identifies each RTP stream
- `rtp.timestamp` — the RTP timestamp, used to synchronize media playback and identify frame boundaries
- `rtp.seq` — the RTP sequence number
- `rtp.cc` — the number of contributing sources (CSRC count)
- `rtp.ext` — a flag indicating whether an RTP header extension is present (0 or 1)
- `rtp.ext.len` — the length of the RTP header extension in 32-bit words

Note that `tshark` will only populate RTP fields for packets that it has decoded as RTP. You may need to filter for RTP packets specifically using the `-Y rtp` display filter.

You may use any preferred language to programatically answer the following questions. The recommended workflow is to use the `pandas` library in Python for writing queries as we will have gone over in discussion section.

Isolating RTP Streams

RTP uses a **Synchronization Source (SSRC)** identifier to distinguish between different media streams within the same session. Each audio or video stream has a unique SSRC, allowing the receiver to separate and reassemble multiple concurrent streams that may be transmitted over the same connection. A Google Meet call will typically produce several RTP streams—some carrying video, some carrying audio, and some in each direction (inbound and outbound from the local client's perspective).

1. Isolate each of the RTP streams in the packet capture, where each stream is identified by its unique SSRC value. Filter out any RTP streams with fewer than 1000 total packets, as these are likely control traffic or transient streams rather than sustained audio or video flows.

Frame Analysis

In RTP, media is transmitted as a continuous stream of packets. For video, multiple packets may be needed to carry a single video frame due to MTU constraints. The RTP **timestamp** field provides the key to grouping packets into frames: all packets belonging to the same video frame share the same RTP timestamp value, and the timestamp changes when a new frame begins.

Once packets are grouped into frames, the size of each frame can be computed from the **RTP payload length**—the number of bytes of actual media data carried in each packet, excluding all header overhead. The RTP payload length can be calculated from the UDP datagram length as follows:

$$\text{RTP payload length} = \text{udp.len} - 8 - 12 - (4 \times \text{rtp.cc}) - (\text{rtp.ext} \times (4 \times \text{rtp.ext.len}))$$

where 8 bytes are subtracted for the UDP header, 12 bytes for the fixed RTP header, and additional bytes are subtracted for any contributing source list and header extensions present in the packet.

2. Compute the RTP payload length for each RTP packet using the formula above.

3. Group the packets for each RTP stream into frames. Recall that all packets belonging to the same frame share the same RTP timestamp value. Compute the size of each frame as the sum of the RTP payload lengths for all packets belonging to that frame.

4. Plot the frame size (y-axis) for each stream against the frame number (x-axis). Display each stream on a separate graph.

5. Based on your graphs from Question 4, comment on which RTP streams are for video and which are for audio. What characteristics of the frame size distribution helped you distinguish between the two?

QoE Metrics

Frames Per Second

Now group the RTP packets for the video streams into one-second windows based on the RTP timestamp values. Use the sampling rate value of `90 kHz` for converting the RTP timestamp values into seconds. The number of distinct RTP timestamps (i.e., distinct frames) within each one-second window gives you the frames per second (FPS) for that window. Note that we are using a different inference method for FPS than the one used in discussion section because we have already estimated the sampling rate, and can more accurately infer the intended FPS using just the RTP timestamps.

6. For each one-second window (x-axis), plot the number of frames per second (y-axis) for each video stream. Display each stream on a separate graph.

Extra Credit

For completing questions 7 and 8 below you will receive extra credit worth 10% of this assignment. This will equate to roughly 1% to your final grade each, for a total of 2% to your final grade if you complete both of them successfully.

Sampling Rate (Optional)

RTP timestamps are measured in units defined by the **sampling rate** of the media stream. For video streams, the sampling rate is nominally `90 kHz` (i.e., 90,000 timestamp units per second), which is a standard value defined in RFC 3551. You can verify this empirically by estimating the sampling rate directly from the packet capture.

To estimate the sampling rate, use the method described in discussion section: compute the ratio of the change in RTP timestamp to the change in wall-clock time (in seconds) across consecutive frames, and take the median or mean of this ratio across all frames in the stream.

7. For the streams that you identified as video in Question 5, estimate the sampling rate using the method described above. What value do you estimate for each RTP video stream? Is it close to `90 kHz`?

Jitter (Optional)

Jitter measures the variability in packet inter-arrival timing and is an important QoE metric for real-time applications. High jitter means packets are arriving with inconsistent delays

relative to their transmission schedule, which can cause audio or video to stutter or drop. The jitter calculation below is taken from Appendix A.8 in RFC 3550:

```
The inputs are r->ts, the timestamp from
the incoming packet, and arrival, the current time in the same units.
Here s points to state for the source; s->transit holds the relative
transit time for the previous packet, and s->jitter holds the
estimated jitter.  The jitter field of the reception report is
measured in timestamp units and expressed as an unsigned integer, but
the jitter estimate is kept in a floating point.  As each data packet
arrives, the jitter estimate is updated:

  int transit = arrival - r->ts;
  int d = transit - s->transit;
  s->transit = transit;
  if (d < 0) d = -d;
  s->jitter += (1./16.) * ((double)d - s->jitter);
```

Both `s->transit` and `s->jitter` can be initialized to 0. You should omit the first jitter estimate when both `s->transit` and `s->jitter` are still 0.

8. Plot the frame-level jitter for each video stream using a sampling rate of `90 kHz`. This means we would use the formula above, but consider the arrival times of entire frames, not individual packets. A frame is considered 'arrived' when the last packet for that frame is delivered. Similarly, `r->ts` would be the RTP timestamp for that frame. Display each stream on a separate graph.

Gradescope

Submissions will made done through Gradescope as a PDF. You can enroll in the course through Gradescope using the entry code **4DWRD5**. Please use either your @ucsb.edu or @umail.ucsb.edu email when joining the course. You may submit a PDF writeup along with code for your submission, or submit a jupyter notebook with the code + answers. Remember to fully evaluate the cells in your jupyter notebook if you choose that option for submission. Late submissions will be accepted and deducted in accordance to the course late-day policy.